Deduce, Propose, Correct: Probability-Accountable LLM Shortlists for Typed Program Synthesis

Tri Nguyen **Thanh-Dat Nguyen**
Harvard University
Basis Research Institute
datnguyen@seas.harvard.edu

Abstract

Large language models can guide program synthesis, but sequential Monte Carlo correction requires an evaluable probability for the proposal actually sampled by the search. We introduce DPC, a typed programming-by-example system that converts four model-proposed repairs—including failures and duplicates—into an application-computed, full-support proposal. In a method-frozen fresh-blind study of 12 finite-domain filter–map tasks, the LLM-guided system solved 10 tasks within 29 logical complete-program slots, versus 2 for a matched grammar-random system; all eight discordant outcomes favored LLM guidance (one-sided exact sign test, $p = 1/256$). However, a separately frozen provider-free calibration study failed its $N = 256$ target-mass gate (RMSE 0.2395; bias +0.1130), despite exact-program discovery in all 128 runs. A post-failure terminal diagnostic verified the finite-state importance identities and supported poor proposal–target overlap as a material error source, without validating the sequential procedure. The evidence therefore supports bounded discovery and auditable proposal accounting, but not calibrated finite-particle inference, general program synthesis, or wall-clock superiority.

Keywords: program synthesis, programming by example, large language models, sequential Monte Carlo, neuro-symbolic inference

1 Introduction

Programming by example (PBE) asks for a program that maps each supplied input to its paired output. The specification is accessible but incomplete: many programs may fit a small example set, and the space of typed recursive programs grows combinatorially. Classical systems control that space with domain languages, deduction, and structural search orders (Gulwani, 2011; Feser et al., 2015); learned policies and LLMs can instead prioritize plausible code (Kalyan et al., 2018; Barke et al., 2024; Li et al., 2024).

The empirical question "did the LLM help?" is easy to answer too broadly. A type checker, execution engine, or selector may account for a system's success, while an LLM supplies only candidates. Conversely, reporting a passing sample without its acquisition and selection rules hides failures and multiplicities. The scientific object should therefore be the complete model–interpreter–SMC system, with the proposal, target, budget, and comparator stated separately.

Sequential Monte Carlo (SMC) supplies a population view of synthesis: programs are particles, semantic fit defines stage targets, and resampling concentrates computation. This view has precedents in neural program synthesis (Ellis et al., 2019) and executable model

discovery (Wahl et al., 2026). Its probabilistic interpretation, however, requires an evaluable proposal probability. Free-form generation does not expose the probability of the canonical abstract syntax tree consumed by an interpreter, especially when invalid responses, duplicate repairs, and textual aliases are present.

DPC avoids that unsupported identification. The model returns four typed repair slots, but the application totalizes those slots and computes the law used for sampling and weighting. An auxiliary-variable construction cancels the unknown provider-response law from the importance weight. This makes the implemented transition probability accountable without claiming to recover an LLM probability over programs.

Our contribution is a bounded, auditable synthesis mechanism and an evidence sequence that keeps discovery distinct from calibration. We formalize typed execution evidence, four-slot failure totalization, a normalized grammar restart, the resulting SMC target and weights, and an online/reference execution boundary. We then report a fresh-blind paired discovery study, a method-frozen negative calibration study, and a separately frozen terminal mechanism diagnostic. A released-data ExeDec adapter study is described only as an integrity-limited debug exercise. Its descriptive outcome is reported with the public-data, repeated-seed, finite-probe, and operations-evidence boundaries kept explicit.

2 Typed Filter–Map Synthesis

Let $\mathcal{D} = \{(x_j, y_j)\}_{j=1}^{M}$ be examples with inferred signature $\tau_{\text{in}} \to \tau_{\text{out}}$. A program e is exact on $\mathcal{D}$ when $e(x_j) = y_j$ for every j. The bounded typed DSL contains integer and Boolean expressions, lists, conditionals, `map`, and right folds. A structural cost favors compact programs, following the minimum-cost perspective of Lambda^2 (Feser et al., 2015). Exactness on examples is not, by itself, evidence of semantics outside the declared evaluation domain.

The confirmatory study uses the strict filter–map skeleton

$$\text{foldr}(\lambda i, a.\ \textbf{if}\ p(i)\ \textbf{then}\ g_m(i) :: a\ \textbf{else}\ a,\ [],\ x), \tag{1}$$

with typed holes $p : \text{Int} \to \text{Bool}$ and $g_m : \text{Int} \to \text{Int}$. A normalized recursive grammar defines a law $g(e)$ over complete programs within the declared bounds. The method does not require the provider to know or enumerate the complete-program support.

Execution supplies sound local evidence. Singleton examples determine facts such as $p(-3) = \text{false}$ or $g_m(2) = 4$ when the alignment is unambiguous; ambiguous examples generate no such fact. For a current program, the harness records predicate and mapper violations, execution loss, the selected typed hole, and legal local grammar. The provider sees only this public evidence and never a hidden target, generator secret, exhaustive catalog, or comparator outcome.

The application is authoritative at every boundary: it parses candidate syntax, checks types, executes programs under fixed resource limits, and canonicalizes states before caching. The provider proposes repairs but does not declare validity, semantic success, or confidence. This division permits a matched grammar acquisition arm to share the interpreter, evidence, target, weighting, and resampling machinery.

3 Probability-Accountable Repair Shortlists

Given current program x, selected hole h, and structured public evidence, the provider is asked for four typed repairs. Every missing, malformed, invalid, or no-op slot is mapped to x, and duplicate multiplicity is retained. If the four totalized slots are $S = (s_1, \dots, s_4)$, the application defines

$$H_S(y) = \frac{1}{4}\sum_{j=1}^{4} \mathbf{1}[s_j = y] \quad \text{and} \quad q(y \mid S, x, h) = (1-\varepsilon)H_S(y) + \varepsilon g(y). \tag{2}$$

The r5, V1, ExeDec V2, and sticky V2 arm use $\varepsilon = 0.05$; terminal V2 additionally evaluates prespecified diagnostic alternatives. Because g is normalized and has full declared grammar support, q is normalized, evaluable, and positive on every grammar program. Retries and backfills are forbidden: a failed response remains four current-state sentinel slots in the fixed-denominator record.

The raw response is an auxiliary variable rather than a claimed program probability. Let $R_t^a(S_t \mid x_{t-1}, h_t, \mathcal{D})$ be the shortlist law induced by acquisition arm a. For the LLM arm this law is unknown; for the grammar-random arm it is weighted sampling without replacement among legal non-current repairs. For the deterministic V1 ranking oracle, R_t^a is a point mass on its mechanically ranked four-tuple. At an exact V1 parent, acquisition is skipped and this tuple is (x, x, x, x); only the shortlist component is absorbing because the εg restart retains full support. Terminal V2 deliberately replaces this sticky shortlist in its diagnostic alternatives. For every arm, the extended target and proposal include the same acquisition factor, so it cancels. For the four-slot method arms, the remaining denominator is exactly Equation 2; terminal V2 alternatives use their separately declared evaluable q, including a 64-slot empirical law for the top-64 control.

This construction is probability-accountable, not automatically efficient. Shortlists that overlap poorly with a target can yield highly variable weights even when every proposal probability is correct. The calibration studies below therefore treat discovery of an exact program and approximation of target mass as separate endpoints.

4 Importance-Corrected SMC

Let $V_p(x)$ and $V_m(x)$ be frozen predicate- and mapper-singleton violation counts, let $L_{\mathcal{D}}(x)$ be the capped soft execution loss, and let $C(x)$ be program cost. The r5 and V1/V2 tasks use $(\lambda_L, \lambda_C, \lambda_E) = (0.75, 0.02, 2)$; ExeDec V2 uses its frozen scorer defaults $(2.0, 0.15, 2)$. Write $\ell(x) = -\lambda_L L_{\mathcal{D}}(x) - \lambda_C C(x)$. The four unnormalized single-program stage densities are

$$\begin{aligned}
\gamma_1(x) &= g(x)\exp[-\lambda_E V_p(x)],\\
\gamma_2(x) &= g(x)\exp[-\lambda_E\{V_p(x) + V_m(x)\}],\\
\gamma_3(x) &= g(x)\exp[-\tfrac{\lambda_E}{2}\{V_p(x) + V_m(x)\} + \tfrac{1}{2}\ell(x)],\\
\gamma_4(x) &= g(x)\exp[\ell(x)].
\end{aligned} \tag{3}$$

These are declared finite computational targets; the soft-loss exponential is not presented as a real-world likelihood or a posterior over unbounded code.

Conditioning on the realized initial program x_0, arm a has extended-path target and proposal

$$\Gamma_t^a(x_{1:t}, S_{1:t}) = \prod_{s=1}^{t} R_s^a(S_s \mid x_{s-1}, h_s, \mathcal{D})\, \gamma_s(x_s),$$
$$Q_t^a(x_{1:t}, S_{1:t}) = \prod_{s=1}^{t} R_s^a(S_s \mid x_{s-1}, h_s, \mathcal{D})\, q(x_s \mid S_s, x_{s-1}, h_s). \tag{4}$$

The shared R_s^a terms cancel, leaving incremental potential

$$G_t(x_{t-1}, S_t, x_t) = \frac{\gamma_t(x_t)}{q(x_t \mid S_t, x_{t-1}, h_t)}. \tag{5}$$

This cancellation does not identify the marginal free-form LLM probability of x_t.

If parent i has normalized weight W_{t-1}^i and produces $B_{t,i}$ children, child b receives unnormalized weight

$$\widetilde{W}_t^{i,b} = \frac{W_{t-1}^i}{B_{t,i}} \frac{\gamma_t(X_t^{i,b})}{q(X_t^{i,b} \mid S_t^i, X_{t-1}^i, h_t^i)}. \tag{6}$$

The factor $1/B_{t,i}$ allocates parent mass across scheduled offspring. After stages 1–3, systematic resampling selects the frozen parent count and resets retained weights. Marginalizing auxiliary shortlists and previous states gives terminal marginal γ_4/Z_4; the product-path normalizer is a different quantity and is not used as a terminal-target calibration claim.

The implementation keeps online synthesis separate from exact reference work. The r5 and ExeDec V2 online accounts cover only scheduled complete-program executions. Both report logical slots, online physical scorer calls, and provider calls; r5 additionally reports cache reuse and tokens. The provider-free V1/V2 studies and ExeDec V2's later public-reference step separately enumerate bounded supports; those reference evaluations are excluded from online work counts. Programs found during reference enumeration are never credited as online discovery.

5 Studies

The evidence has three ordered layers. The r5 study tests bounded discovery by the complete LLM-shortlist SMC system against a matched grammar-shortlist arm. Calibration V1 tests finite-particle approximation against exact references and retains a failed gate. Terminal diagnostic V2 probes a mechanism for that failure without revising it. A fourth, clearly separated ExeDec V2 subsection records an adapter debug protocol whose public-data and custody limitations preclude an efficacy claim.

5.1 Fresh-blind r5 paired discovery

The r5 method was frozen before a new custodial secret generated 12 independently drawn targets from the rejection-conditioned recursive filter–map grammar. Each public task contained complete singleton coverage of the declared integer domain $\{-3, \ldots, 4\}$ plus list examples. The provider was GPT-OSS-120B at a sealed revision (OpenAI, 2025). Provider

calls in r5 and ExeDec V2 shared frozen settings: temperature zero, low reasoning effort, a 1,200-token output cap, a 420-second timeout, concurrency two, and no retries. Private target material remained outside the provider environment until public execution, reveal-free analysis, and public sealing were complete.

For each task, grammar-random ran before the LLM arm. Both used the same initial law, evidence, hole policy, four-slot totalization, grammar restart, stage targets, importance weights, resampling, and 29-slot trajectory. They differed only in shortlist acquisition. Task-specific start, proposal-mixture sampling, and resampling seeds were paired; only acquisition seeds differed. Starting from one executed program, stage 1 drew four children; stages 2–4 drew four children from each of two resampled parents, yielding $1+4+3\times 8 = 29$ logical slots and eight terminal weighted particles. The primary endpoint was zero public-example loss by slot 29; complete singleton coverage made that endpoint equivalent to itemwise filter–map behavior on the declared domain, which the post-unblind audit verified. The task was the statistical unit, and the frozen analysis used a one-sided exact sign test over discordant outcomes. All trajectories ran the full budget without outcome-dependent stopping.

5.2 Calibration V1 and terminal diagnostic V2

Calibration V1 was frozen separately and used no provider. Four developmental filter–map tasks each had an exactly enumerable 36,000-program support. A deterministic public-evidence ranking oracle supplied four local slots and was mixed with the same grammar restart; it was an exhaustive diagnostic component, not an LLM or a search baseline. The study used $N \in \{32, 64, 128, 256, 512\}$ and 32 repetitions per task–N cell. Here N is the terminal weighted-particle count; each repetition scheduled $1 + 4N$ logical slots and did not stop early. Its frozen $N = 256$ gate required pooled exact-program-mass RMSE below 0.10 and signed bias inside $[-0.03, 0.03]$.

After V1 failed, terminal diagnostic V2 was frozen before new Monte Carlo draws. It checked the finite-state importance identities and compared the sticky V1 terminal proposal with local alternatives and an exhaustive global top-64, $\varepsilon = 0.50$ positive control. It evaluated six arms at $N \in \{256, 512\}$ with 128 repetitions per task–arm–N cell. The diagnostic conditions on a mechanically selected zero-loss parent and addresses only the terminal calculation. It cannot rescue V1, validate the four-stage recurrence, or establish online search efficiency.

5.3 ExeDec V2 released-data debug

ExeDec V2 is a descriptive debug study over four deduplicated targets from the released ExeDec DeepCoder data (Shi et al., 2024). The local adapter accepts only a strict Filter-then-Map subset and collapses each two-line source program to the skeleton in Equation 1; "DeepCoder-HO" is a local label, not an official ExeDec split. The design pairs eight seeds per task across LLM-SMC and grammar-random, giving 32 paired blocks and 64 scheduled runs. Its endpoint is exactness on finite private debug probes, with an exact two-sided discordant-pair test and no efficacy gate.

The targets and released examples are public and potentially present in model training data. The private probes are finite debug checks, not fresh hidden generalization or a proof of semantic equivalence. The hash-bound shared-FUSE staging tree was not an

exclusive operating-system custody boundary. The 64-run study archive was independently verified and imported, but the separate operations-evidence export failed its frozen safe-mode verifier because tar header modes violated the sealed contract. Independent audit found its content and cross-bindings valid; the original failure remains preserved. A local mode-header-normalized derivative was validated but is noncanonical and was not imported. The result is therefore debug-only and integrity-limited.

6 Results

We report the findings in the same order as the study designs and do not pool bounded discovery with target-mass calibration or adapter debugging.

6.1 Fresh-blind r5 discovery

The r5 LLM arm found a finite-domain exact program by slot 29 on 10 of 12 tasks, versus 2 of 12 for grammar-random. There were eight LLM-only successes, no grammar-random-only successes, and two tasks solved by both arms. Thus the paired advantage was eight. The one-sided exact sign-test value was $p_{\text{sign}} = 2^{-8} = 1/256 = 0.00390625$. As a post hoc sensitivity that treats the observed-but-invalid r4 diagnostic as an additional numerical look, a two-look Bonferroni adjustment gives 0.0078125. This is not the frozen r5 analysis: r3 had no numerical outcome, and r4 remains invalid and excluded. The frozen thresholds were at least 6 LLM successes and at least a four-task advantage; both were met. At the descriptive slot-21 checkpoint, the counts were 8 of 12 and 2 of 12. These data compare the complete LLM-shortlist system with its matched grammar-shortlist arm; they do not isolate the model from shared execution, weighting, or resampling.

The 24 trajectories executed 696 logical program slots. Scorer caching reduced this to 367 physical scorer calls, with 329 cache hits. The LLM arm made 58 provider calls and used 137,759 tokens; grammar-random made no provider calls. Two responses ended at the 1,200-token limit and, as frozen, each was totalized to four current-state sentinel slots without retry or backfill.

Reveal-free analysis and a 415-file public checksum inventory were sealed before unblinding. The audit then verified the secret and target commitments, reproduced all 12 public tasks, and re-executed every hidden target. Across the 12 successful task–arm trajectories, four first-exact programs were syntax-identical to the target and eight were semantic aliases. Because public singletons covered the complete declared domain, this is finite-domain semantic recovery, not unique-AST or out-of-domain recovery.

6.2 Negative calibration and terminal diagnosis

V1 failed both frozen $N = 256$ gate components despite exact-program discovery in all 128 task–repetition runs (Table 1). The result directly separates the ability to encounter an exact program from the ability to estimate the target mass assigned to exact programs.

Table 1: Frozen provider-free V1 calibration gate at $N = 256$, pooled over four tasks and 32 repetitions per task. Discovery was descriptive.

Endpoint	Criterion	Observed	Decision
Exact-mass RMSE	< 0.10	0.2395265	Fail
Signed exact-mass bias	$[-0.03, 0.03]$	+0.1130184	Fail
Zero-loss discovery	descriptive	128/128	Not gated

For $N = 32, 64, 128, 256, 512$, pooled exact-mass RMSE was 0.32468, 0.26776, 0.26536, 0.23953, and 0.25592, respectively. This grid does not support a reference $N^{-1/2}$ error rate. Exact evaluation of q was necessary for correction but insufficient for the frozen finite-budget calibration target under this proposal, schedule, and four-task suite.

In terminal diagnostic V2, the finite-state importance identities held to a maximum absolute error of 2.22×10^{-16}. The exhaustive public-score global top-64, $\varepsilon = 0.50$ positive control had a larger exact population ESS fraction than the sticky proposal on every task and reduced pooled $N = 512$ exact-mass RMSE from 0.2792 to 0.0626. The frozen mechanism-support rule was met, supporting poor proposal–terminal-target overlap as a material source of error in the tested terminal step. Because the control exhaustively scores the finite support and conditions on a zero-loss parent, it is not a search algorithm and does not revise V1's failed gate.

6.3 ExeDec V2 debug

All 64 runs formed 32 complete paired seed blocks. LLM-SMC was exact on every stored private debug probe in 17/32 blocks, versus 3/32 for grammar-random, a success-fraction difference of 0.4375. There were 16 LLM-only successes, two grammar-random-only successes, one success in both arms, and 13 in neither arm, so the 18 discordant blocks gave an exact two-sided conditional sign-test value of $p = 0.001312255859375$. This value is descriptive and is not treated as a statistical-significance or confirmatory claim: eight repeated seeds are nested within each of only four public released targets, and the protocol specified no efficacy gate.

Public-example exactness was 21/32 for LLM-SMC and 5/32 for grammar-random. The runs used 1,856 logical program slots and 1,040 online physical scorer calls before reference enumeration; exhaustive public-reference evaluations are excluded from that count. LLM-SMC made 188 provider calls under the frozen aggregate cap of 224; grammar-random made none. Conditional on hidden-probe success, the mean first success slot was 12 for LLM-SMC and 9 for grammar-random; mean terminal ESS out of eight particles was 4.7512 and 3.0567, respectively. The conditional slot summary is not a speed comparison, and passing finitely many private probes does not establish semantic equivalence.

Table 2: ExeDec V2 hidden-probe exact successes by released target. Each row contains eight repeated paired seed blocks; these four targets, not 32 fresh tasks, define the limited target diversity.

Released target	LLM-SMC	Grammar-random
`exedec-debug-0131`	2/8	0/8
`exedec-debug-0245`	5/8	1/8
`exedec-debug-0258`	4/8	1/8
`exedec-debug-0608`	6/8	1/8
Overall	17/32	3/32

The canonical study analysis passed its internal integrity checks, including validation of all public run artifacts before private debug oracles were opened. The auxiliary operations-evidence packaging failure nevertheless remains part of the result's provenance: neither its valid content audit nor a noncanonical mode-header derivative creates an exclusive custody boundary or turns this released-data adapter exercise into efficacy evidence.

7 Related Work

Symbolic PBE systems use domain languages, deduction, and structural biases to control ambiguity (Gulwani, 2011; Feser et al., 2015). Neural guided deduction and LLM-guided synthesis instead learn or prompt search priorities (Kalyan et al., 2018; Barke et al., 2024; Li et al., 2024). DPC shares their hybrid view but makes a narrower contribution: the application turns a fixed repair shortlist into an explicit, replayable sampling law and compares shortlist acquisition inside otherwise shared mechanics.

SMC and related particle methods provide standard tools for sequential importance weighting and resampling (Del Moral, 2004; Del Moral et al., 2006; Naesseth et al., 2019). Prior program and model synthesis work already uses learned guidance and particle inference (Ellis et al., 2019; Wahl et al., 2026). We do not claim the first use of SMC for synthesis. The distinction here is the totalized four-slot auxiliary-variable proposal and an empirical program that explicitly tests, and fails, finite-N calibration rather than inferring it from exact proposal accounting.

ExeDec studies execution decomposition across released DeepCoder and RobustFill tasks (Shi et al., 2024). Our narrow adapter neither implements the complete DeepCoder language nor reproduces ExeDec's official split structure. It is included only to expose adapter behavior and accounting on released examples before any future external confirmation.

8 Limitations and Integrity

The r5 result covers 12 generated filter–map tasks over eight integer values with public singleton examples that identify behavior on that domain. It uses one pinned model revision and one matched acquisition comparator. The tasks are not representative of arbitrary

PBE, the result provides no extrapolation beyond the observed domain, and temperature-zero calls are not independent model replicates. Exact references in calibration are available only because the four developmental supports are small enough to enumerate.

The calibration evidence is intentionally negative. V1's failed gate remains failed; terminal V2 identifies one material mechanism only within a conditioned, provider-free terminal calculation. Neither exact proposal accounting nor high discovery establishes low-variance importance weights, calibrated posterior mass, an unbiased log normalizer, or validity of the full sequential approximation.

The ExeDec study is weaker still: it uses four deduplicated released targets, eight repeated seeds within each target, an unofficial strict Filter-then-Map adapter, potentially contaminated public data, and finite debug probes. The 32 blocks are therefore not 32 independent targets, and the exact test value is descriptive rather than a significance or confirmation claim. Its frozen scorer scales also differ from r5, so it is not a scale-matched external confirmation. Nonexclusive FUSE staging and the failed original operations-evidence safe-mode verification add an integrity limit that the noncanonical local derivative does not remove. More generally, equal logical slots do not imply equal wall time, provider compute, or total compute; the studies establish no speedup over enumeration or other synthesizers. Generated programs require sandboxing and review. Appendix C preserves the reported-study failure history.

9 Conclusion

DPC turns four typed LLM repair slots into an application-computed, full-support proposal law and uses that law in an explicit SMC correction. In a fresh-blind bounded study, the complete LLM-shortlist mechanism solved 10 of 12 tasks versus 2 of 12 for matched grammar acquisition. This supports a narrow discovery advantage for the tested system, not an LLM-only, speed, or broad generalization claim.

The accompanying calibration result is equally central: an exactly evaluable proposal and discovery in every $N = 256$ run did not produce an accurate finite-particle target-mass estimate. The terminal diagnostic makes poor proposal–target overlap a concrete mechanism for future work without erasing the failed gate. Stronger evidence now requires a revised, freshly calibrated proposal and independently confirmed external tasks, with discovery, distributional accuracy, and compute kept as separate endpoints.

Acknowledgments and Disclosure of Funding

This draft received no declared external funding. Compute and hardware metadata remain under author review. The final submission must include complete funding, compute, competing-interest, and author disclosures.

Appendix A. Proposal Normalization

For the four-slot method arms, totalization guarantees that S has exactly four slots, so H_S in Equation 2 sums to one even if every provider slot fails or all four slots are duplicates. The recursive grammar law g also sums to one. Their convex mixture is therefore normalized, and

because $\varepsilon > 0$, every grammar program receives positive mass. The V2 top-64 control uses the analogous normalized 64-slot empirical law. In the extended target and proposal, the same normalized acquisition factor R_t^a appears at each stage and cancels from Equation 5. These statements prove the corresponding implemented proposal laws; they do not imply finite-N calibration.

Appendix B. Task-Level Results

Table 3 reports every task–arm trajectory in the sealed r5 analysis. A provider-totalized slot is either an explicit no-op mapped to the current state or a sentinel produced by whole-response totalization; grammar-random made no provider calls.

Table 3: Fresh-blind r5 task-level outcomes. Success is zero public-example loss by slot 29; "–" means no exact discovery or no applicable provider quantity.

Task	Arm	Success	First exact	Best loss	Totalized slots
`blind-v3-01`	Grammar	No	–	12	–
`blind-v3-01`	LLM	Yes	29	0	1
`blind-v3-02`	Grammar	No	–	10	–
`blind-v3-02`	LLM	Yes	19	0	1
`blind-v3-03`	Grammar	No	–	11	–
`blind-v3-03`	LLM	Yes	22	0	5
`blind-v3-04`	Grammar	No	–	22	–
`blind-v3-04`	LLM	Yes	11	0	0
`blind-v3-05`	Grammar	No	–	17	–
`blind-v3-05`	LLM	Yes	14	0	0
`blind-v3-06`	Grammar	Yes	18	0	–
`blind-v3-06`	LLM	Yes	14	0	1
`blind-v3-07`	Grammar	No	–	20	–
`blind-v3-07`	LLM	No	–	20	2
`blind-v3-08`	Grammar	No	–	11	–
`blind-v3-08`	LLM	No	–	20	4
`blind-v3-09`	Grammar	No	–	15	–
`blind-v3-09`	LLM	Yes	6	0	0
`blind-v3-10`	Grammar	No	–	10	–
`blind-v3-10`	LLM	Yes	9	0	0
`blind-v3-11`	Grammar	No	–	12	–
`blind-v3-11`	LLM	Yes	9	0	0
`blind-v3-12`	Grammar	Yes	16	0	–
`blind-v3-12`	LLM	Yes	8	0	0

Table 4 exposes the task heterogeneity behind the pooled V1 failure. RMSE and bias summarize 32 self-normalized exact-mass estimates for each task at $N = 256$.

Table 4: Provider-free V1 calibration by task at $N = 256$. Discovery was descriptive and not a gate component.

Task	Reference mass	RMSE	Bias	Discovery	Mean ESS/N
`calibration-equality-scale`	0.1101251	0.2045285	−0.0501036	32/32	0.0173374
`calibration-lower-shift`	0.8988287	0.0960045	+0.0957645	32/32	0.9186058
`calibration-upper-negate`	0.9208953	0.0753309	+0.0662831	32/32	0.8776247
`foldr-bounded-square`	0.4944417	0.4156541	+0.3401296	32/32	0.4436003

Appendix C. Preserved Integrity and Failure Timeline

The following chronology is part of the evidence rather than a second results narrative. Blind-confirmation revisions r1 and r2 were superseded before secret preparation. R3 aborted before program execution or provider calls because a derived run record omitted two harness-required fields. R4 completed public execution and reveal-free analysis but failed its frozen sealing gate before unblinding; it remains blinded and excluded. Its invalidated reveal-free diagnostic was 11/12 versus 2/12 at slot 29 and 10/12 versus 0/12 at slot 21; these values are preserved only to disclose the revision path and are not evidence. Before any new secret or call, r5 changed only the sealer contract and focused test, retained r4's scientific execution method, and generated a fresh secret and suite.

Provider-free calibration V1 failed and remains failed. Terminal V2 was frozen after that failure and is diagnostic only. ExeDec benchmark V1 was superseded before provider calls because its source closure omitted import-time files. ExeDec V2 is the final debug protocol. Its 1,143-file study archive passed the frozen transfer verifier and was atomically imported; the canonical analysis then passed its own run-integrity checks.

The separate original ExeDec V2 operations-evidence archive failed its frozen safe-mode verifier at `files/operations/bin/exedec_v2_driver.py`. All 149 manifested content hashes and scientific/operational cross-bindings independently validated, but the shared-FUSE tar-header modes violated the sealed packaging contract, so the original failure remains authoritative. A separately audited local derivative changed only the 150 tar mode headers to 146 files at `0600` and four executables at `0700`. That derivative and its report and seal are explicitly noncanonical and were not imported. Public-data, adapter, finite-probe, contamination, and nonexclusive-custody limits therefore apply to every report. Frozen protocols, failed transfers, and artifact directories remain in place rather than being rewritten to fit the final narrative.

Appendix D. Exact-Reference Claim Boundary

Reference enumeration determines whether exact programs exist while computing target functionals. It therefore supports probability-accounting and estimator diagnostics, not faster synthesis. The V1 local ranking oracle and the V2 global top-64, $\varepsilon = 0.50$ control both use exhaustive public-score information unavailable to the online r5 search. Their discoveries, ranks, and work are never credited to the LLM or SMC search, and the positive control is not a practical synthesis algorithm.

Appendix E. Artifact Availability

The data-and-paper companion is available in the public Hugging Face archive. Implementation source and tests are available from the GitHub publication branch; the completed-study revision is `d0eacd1e6315cceb640064c6b9a5fc5919ec05a5`.

R5 protocol SHA-256: `36bd14082c767f47327b73a1ee57f44763fcd97b2bea6dcc a0637a8cddc83d2d`.

R5 manifest: `artifacts/blind-filter-map-confirmation-v3-r5/public-bundle-seal/SHA256SUMS`.

Calibration V1: `artifacts/particle-calibration-v1/analysis.json`.

Terminal diagnostic V2: `artifacts/particle-calibration-terminal-diagnostic-v2/analysis.json`.

ExeDec V2 protocol SHA-256: `305cd2d89fbe0918a52b8e0ea6b3c4dfce23ad04 ae669c48bcc3fcaa37c15b7d`.

ExeDec V2 imported study tree (1,143 files): `artifacts/exedec-deepcoder-ho-debug-benchmark-v2`. Study archive SHA-256: `5e97831610852264f686d0f37d6f9c4aef8d783e45c29e6cc3fd5023dbc7e46c`.

ExeDec V2 analysis: `artifacts/exedec-deepcoder-ho-debug-benchmark-v2/analysis/analysis.json`. SHA-256: `5bc9a74a39add1a60d8d5adedad89beb9dad389bd4e6cbdb4447b45e 1888c2bb`.

ExeDec V2 analysis inventory: `artifacts/exedec-deepcoder-ho-debug-benchmark-v2/analysis/inventory.json`. SHA-256: `848e204685c8b4459bb51c7471d5c644464f3183af142f6cecc60feb85a65b25`.

Operations-evidence failure record: `artifacts/exedec-deepcoder-ho-debug-benchmark-v2-operations-evidence-failed-v1/README.md`. The rejected original archive and seal have SHA-256 values `aa5b9fa4 a8b60ab318030cfb3ff98882de5889b5200a96ffb1e6ca85a8d96b0a` and `72cc4bb3 998380eb12f78e15b7e17f04b6b676adf059b5efe6521ef804150ebe`.

Noncanonical, unimported mode-header derivative archive/report/seal SHA-256: `cbb65b605b5212f521558249fe2036d8820b88056a4750ea280e5775e1f14393`; `49e5f747b5dd78bde1e3a008205c139b5ad2ff8300602b3ede327db08af050d3`; and `ebd74c77439879be78983bc98bfa7d5301cf30c7d3fb8ad4850f4d63926e3884`. These files remain under `artifacts/exedec-deepcoder-ho-debug-benchmark-v2-operations-evidence-failed-v1/mode-header-normalized-derivative-v1`.

The companion archive distributes data, checksums, failure records, and the manuscript; implementation source is distributed through the GitHub revision above. The path locators in this appendix are relative to that source checkout and its companion artifact tree.

References

Shraddha Barke, Emmanuel Anaya Gonzalez, Saketh Ram Kasibatla, Taylor Berg-Kirkpatrick, and Nadia Polikarpova. HYSYNTH: Context-free LLM approximation for guiding program synthesis. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://openreview.net/forum?id=5jt0ZSA6Co.

Pierre Del Moral. *Feynman-Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer, 2004. doi: 10.1007/978-1-4684-9393-1.

Pierre Del Moral, Arnaud Doucet, and Ajay Jasra. Sequential monte carlo samplers. *Journal of the Royal Statistical Society: Series B*, 68(3):411–436, 2006. doi: 10.1111/j.1467-9868.2006.00553.x.

Kevin Ellis, Maxwell Nye, Yewen Pu, Felix Sosa, Joshua Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a REPL. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://arxiv.org/abs/1906.04604.

John K. Feser, Swarat Chaudhuri, and Isil Dillig. Synthesizing data structure transformations from input-output examples. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 229–239. ACM, 2015. doi: 10.1145/2737924.2737977.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 317–330. ACM, 2011. doi: 10.1145/1926385.1926423.

Ashwin Kalyan, Abhishek Mohta, Oleksandr Polozov, Dhruv Batra, Prateek Jain, and Sumit Gulwani. Neural-guided deductive search for real-time program synthesis from examples. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=rywDjg-RW.

Yixuan Li, Julian Parsert, and Elizabeth Polgreen. Guiding enumerative program synthesis with large language models. *arXiv preprint arXiv:2403.03997*, 2024. doi: 10.48550/arXiv.2403.03997.

Christian A. Naesseth, Fredrik Lindsten, and Thomas B. Schön. Elements of sequential monte carlo. *Foundations and Trends in Machine Learning*, 12(3):187–306, 2019. doi: 10.1561/2200000074.

OpenAI. gpt-oss-120b model documentation. OpenAI Developers documentation, 2025. URL https://developers.openai.com/api/docs/models/gpt-oss-120b. Accessed 2026-08-13.

Kensen Shi, Joey Hong, Yinlin Deng, Pengcheng Yin, Manzil Zaheer, and Charles Sutton. ExeDec: Execution decomposition for compositional generalization in neural program synthesis. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=oTRwljRgiv.

Stefan Wahl, Raphaela Schenk, Ali Farnoud, Jakob H. Macke, and Daniel Gedon. A probabilistic framework for LLM-based model discovery. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://arxiv.org/abs/2602.18266.